



RV College of Engineering®

Mysore Road, RV Vidyaniketan Post, Bengaluru - 560059, Karnataka, India

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

Adaptive Soil Fertility Monitoring System Using Edge Intelligence

IoT and Embedded Computing – CS344AI

Experiential Learning

REPORT

Submitted by:

Raghottam Girish Nadgoudar	1RV24CS214
Praneel K A	1RV24CS200
Rohan S	1RV24CS232

Under the guidance of

Dr. Mohana

Affiliation

In partial fulfilment for the award of degree of

Bachelor of Engineering

in

Department of Computer Science and Engineering

2026-2027

RV COLLEGE OF ENGINEERING® BENGALURU-59

(Autonomous Institution Affiliated to VTU, Belagavi)

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING



Certificate

Certified that the **IoT and Embedded Computing** Experiential Learning titled “*Adaptive Soil Fertility Monitoring Using Edge Intelligence*” is carried out by in partial fulfilment for the requirement of degree of **Bachelor of Engineering in Computer Science and Engineering** of the Visvesvaraya Technological University, Belagavi during the year 2026- 2027. It is certified that all corrections/suggestions indicated for the Internal Assessment have been incorporated in the report.

Dr. Mohana,
Associate Professor,
Department of CSE

Dr. Shanta Rangaswamy,
Professor and HoD,
Department of CSE

External Viva

Name of Examiners

- 1.
- 2

Signature with Date

Contents

1	Abstract	2
2	Problem Statement	2
2.1	Limitations of Existing Soil Monitoring Systems	3
2.2	Challenges	3
2.3	Proposed Solution	4
3	Literature Review	4
4	Objectives	5
5	Methodology	6
5.1	System Architecture Overview	6
5.2	Hardware Development Phase	7
5.2.1	Soil Sensor Interface (RS485 Modbus)	8
5.2.2	MAX485 Communication Module	8
5.2.3	ESP32 Microcontroller	8
5.2.4	Circuit Connections	8
5.3	Data Collection Phase	9
5.4	Data Processing and Feature Engineering	10
5.5	Drift Detection Mechanism	10
5.6	Adaptive Calibration Mechanism	10
5.7	Machine Learning Model Development	11
5.8	Model Deployment on ESP32	11
5.9	Fertilization Event Tracking	12
5.10	Communication Architecture	12
5.11	Software Tools and Visualization Dashboards	13
6	Results and Analysis	14
7	Conclusion	17
8	References	17

1 Abstract

This project presents the design and implementation of an Adaptive Soil Fertility Monitoring System using Edge Intelligence, aimed at enabling real-time, data-driven agricultural decision-making. The system utilizes an ESP32-based embedded platform integrated with an RS485 soil sensor module to continuously monitor key soil parameters, including nitrogen (N), phosphorus (P), potassium (K), moisture, temperature, and electrical conductivity. Unlike conventional soil monitoring solutions that rely on static threshold-based evaluation or cloud-dependent analysis, the proposed system performs all critical computations locally on the edge device, ensuring low latency, reduced communication overhead, and independence from continuous internet connectivity.

The system incorporates a lightweight Cumulative Sum (CUSUM) drift detection algorithm to identify temporal changes in soil fertility trends, enabling early detection of nutrient depletion before critical thresholds are reached. Additionally, an adaptive site calibration mechanism is implemented to dynamically establish baseline fertility levels specific to each deployment location, thereby improving classification accuracy across diverse soil types. A machine learning model, trained offline using scikit-learn and deployed on the ESP32 using m2cgen, classifies soil fertility into distinct categories based on both raw sensor inputs and derived features such as drift score and time since fertilization. Furthermore, the system includes a fertilization event logging feature that tracks post-application nutrient response and evaluates fertilizer effectiveness in real time.

The overall architecture follows a modular pipeline consisting of sensor data acquisition, edge-based feature extraction, drift analysis, adaptive calibration, and on-device machine learning inference. Results are displayed locally on an SSD1306 OLED interface and transmitted via MQTT to a Node-RED gateway, which acts as a bridge to synchronize live telemetry with Firebase Realtime Database. The mobile client interface is built as a cross-platform Expo React Native application, incorporating persistent real-time listeners for live updates. Furthermore, the application integrates a voice-enabled AI Chat Assistant powered by Groq APIs (Whisper and Llama-3.3-70b). This assistant transcribes user speech, handles voice-activated monitoring commands, and dynamically compiles historical soil data and AI-generated insights into professional, downloadable PDF reports using expo-print and expo-sharing. By combining edge intelligence, cloud synchronization, and generative AI, the proposed system provides a robust, context-aware, and highly interactive soil intelligence solution for precision agriculture.

2 Problem Statement

Efficient soil fertility management is a critical requirement in modern agriculture to ensure optimal crop yield and sustainable resource utilization. In many practical scenarios, farmers rely on intuition, periodic observation, or infrequent laboratory testing to assess soil health. These approaches are often inadequate, as they fail to provide continuous insights into soil nutrient dynamics and do not capture temporal variations in soil conditions. As a result,

fertilizer application decisions are frequently suboptimal, leading to over-fertilization, under-fertilization, increased input costs, and potential environmental degradation.

2.1 Limitations of Existing Soil Monitoring Systems

Existing soil monitoring solutions exhibit several limitations:

- Most systems provide only instantaneous sensor readings without considering temporal trends or patterns in soil fertility.
- Fixed threshold-based classification fails to account for variability in soil characteristics across different locations, resulting in inaccurate assessments.
- Conventional systems lack the capability to detect early-stage nutrient depletion, as alerts are triggered only after critical thresholds are crossed.
- Absence of mechanisms to evaluate the effectiveness of fertilizer application, leaving farmers unaware of whether corrective actions have produced the desired outcome.
- Heavy reliance on cloud-based processing in some systems leads to latency, increased communication overhead, and dependency on continuous internet connectivity.

2.2 Challenges

Designing an intelligent soil fertility monitoring system involves several challenges:

- Accurate acquisition and interpretation of multiple soil parameters such as NPK, moisture, temperature, and electrical conductivity.
- Detection of gradual changes and trends in soil fertility rather than relying solely on static values.
- Adapting the system to site-specific soil conditions without requiring manual calibration or expert intervention.
- Implementing computationally efficient algorithms suitable for resource-constrained edge devices such as ESP32.
- Integrating machine learning models into embedded systems while ensuring low latency and minimal resource usage.
- Providing meaningful feedback regarding fertilizer effectiveness based on observed changes in soil parameters over time.

2.3 Proposed Solution

To address the above limitations and challenges, this project proposes an Adaptive Soil Fertility Monitoring System using Edge Intelligence. The proposed solution:

- Continuously monitors key soil parameters using an RS485-based sensor interfaced with an ESP32 microcontroller.
- Implements a CUSUM-based drift detection algorithm to identify early-stage changes in soil fertility trends.
- Employs an adaptive calibration mechanism to dynamically establish baseline conditions specific to each deployment environment.
- Integrates a machine learning classifier deployed on-device to categorize soil fertility levels based on both raw and derived features.
- Incorporates a fertilization event logging mechanism to track nutrient response and evaluate the effectiveness of applied fertilizers.
- Hybrid Communication and Real-Time Sync: Employs a local MQTT broker to handle low-latency transmission from the ESP32 to a Node-RED gateway. Node-RED bridges it to Firebase Realtime Database via REST APIs.
- Voice-Enabled AI Companion: Incorporates a voice-enabled AI assistant in the mobile application.
- Dynamic PDF Reporting and Sharing: Replaces static, complex mobile-rendered charts with a printable, high-fidelity PDF document generated on-demand, when triggered by a voice command or button click.

3 Literature Review

People are getting better at growing crops with the help of precision agriculture. This is because of technologies like IoT, machine learning and embedded systems [1], [2], [7], [14]. These technologies help us keep an eye on the soil and make sure crops are healthy. Researchers have developed tools to measure soil nutrients, moisture and temperature all the time [1], [2], [6]. This helps farmers make decisions about their crops.

A. Soil Nutrient and Moisture Sensing

The older ways of testing soil is not good for checking the soil all the time. To fix this problem people have made Internet of Things systems that can measure soil things like moisture, temperature and nutrients in time [1], [2], [6]. One kind of sensor that measures soil moisture is very popular because it is cheap and does not get damaged easily [5], [11], [15]. There are also sensors that can measure nutrients in the soil and talk to other devices from far

away [8], [12]. Many farming systems use these sensors and wireless communication to make farming easier and more automatic [6], [7], [14]. However most of these systems just collect data and store it in the cloud they do not do much with the data at the place where it is collected [2], [10]. Precision agriculture and Internet of Things are important, for making crops healthier and for automated monitoring of soil health and crop productivity [1], [7], [14].

B. Statistical Drift Detection in Agricultural Monitoring

The soil fertility changes over time because of things like irrigation and rainfall and fertilizer and the crops using up the nutrients. This means that the old ways of checking for nutrient depletion may not work well. There is a need to find a way to see when the nutrients are getting depleted. One way to do this is to use something called the Cumulative Sum algorithm [3], [4]. This algorithm is good at finding changes in the data and it does not need a lot of computer power to work. People have been using this algorithm for a while now [4]. It has been working well. Some new studies have even made it better by making it update the reference statistics all the time [3]. This makes it work better in environments that are always changing. This is really good for systems that do not have a lot of power to spare.

C. Edge Intelligence and Machine Learning for Precision Agriculture

People have been using machine learning in agriculture for things [7], [9], [13], [14]. For example machine learning helps us figure out what kind of soil we have. It also helps us decide what crops to plant. Machine learning can even find diseases in plants. Predict how much food we will get. We use algorithms like Random Forests and Neural Networks [7], [13]. These algorithms work well when we use them with data about the soil and the environment. They help us make decisions and help us understand the soil and the environment better. Although many machine learning systems rely on cloud-based processing [1], [2], there are new developments in Edge Intelligence. These developments allow us to use models directly on devices like the ESP32 [10], [13]. This is good because it reduces the time it takes to communicate and make decisions. It enables us to make decisions locally. Edge Intelligence and machine learning are very useful, for precision agriculture. The proposed system uses Edge Intelligence and machine learning. It combines internet-based sensing and adaptive CUSUM drift detection [3], [4]. It also uses an embedded Random Forest classifier for real-time soil fertility assessment [7], [13]. This means we can use machine learning and Edge Intelligence to check the soil fertility in time.

4 Objectives

The primary objectives of this project are as follows:

- To design and implement an Adaptive Soil Fertility Monitoring System using Edge Intelligence for real-time analysis of soil health parameters.
- To develop a system capable of continuously monitoring key soil attributes, including nitrogen (N), phosphorus (P), potassium (K), moisture, temperature, and electrical conductivity, using embedded sensor modules.

- To implement a lightweight drift detection mechanism based on the CUSUM algorithm for identifying early-stage changes in soil fertility trends.
- To design an adaptive site calibration mechanism that automatically establishes baseline soil conditions specific to each deployment location.
- To develop and deploy a machine learning model on an edge device (ESP32) for classification of soil fertility levels based on both raw sensor data and derived features.
- To incorporate a fertilization event logging system that tracks post-application nutrient response and evaluates fertilizer effectiveness.
- To design an efficient edge-based processing pipeline that minimizes reliance on cloud infrastructure and ensures low-latency decision-making.
- To implement a secure, firewall-resilient real-time synchronization layer using a Node-RED bridge and Firebase Realtime Database to stream processed soil data to cross-platform mobile clients.
- To ensure the system is scalable, modular, and suitable for practical agricultural deployment, supporting precision farming applications.
- To integrate an AI-powered voice assistant in the mobile app using Groq Whisper for transcribing voice queries, providing real-time soil health recommendations, generating and sharing comprehensive PDF soil analysis reports.

5 Methodology

5.1 System Architecture Overview

The proposed Adaptive Soil Fertility Monitoring System is designed as a modular, edge-based architecture that performs real-time data acquisition, processing, and intelligent decision-making directly on the embedded device. The system integrates sensor interfacing, feature extraction, drift detection, adaptive calibration, and machine learning-based classification into a unified pipeline executed on an ESP32 microcontroller. Raw Modbus RTU frames from the RS485 soil sensor are decoded into six numerical parameters: nitrogen (N), phosphorus (P), potassium (K), moisture, temperature at a fixed sampling interval of 30 seconds. These values are passed to the feature extraction stage, which appends two derived features: the current CUSUM statistic and elapsed time since the last logged fertilization event, producing an 7-dimensional input vector. This vector is consumed by the adaptive calibration engine, which maintains and updates the site-specific baseline, and subsequently by the on-device ML inference module. Classification results and drift alerts are dispatched concurrently to the OLED display and the MQTT publisher. The MQTT publisher broadcasts to the topic soil/readings on a local broker.

A Node-RED bridge subscribes to this topic, processes the JSON payloads, and synchronizes the state to the cloud. It performs an HTTP PUT request to update the latest reading (/soil/readings/latest.json) and an HTTP POST request to append to the historical record (/soil/readings/history.json).

The cross-platform mobile application (SoilMonitorApp) connects to the Firebase Realtime Database using the Firebase JS SDK. By establishing persistent listeners (onValue), the app updates its state reactively as new readings arrive.

The application UI is structured around four primary screens: Status (live parameters and alerts), Trends (interactive dashboards), Settings (configurations and API credentials), and the AI Chat Tab (voice-enabled interface). The AI Chat Tab introduces a natural language processing loop: local voice inputs are captured, transcribed using Groq Whisper, parsed for custom commands, and processed by Groq Llama-3.3-70b with full context injection (current readings, trends, and calibration baselines) to output real-time agronomic insights and generate downloadable PDF reports.

The overall system flow is illustrated below:

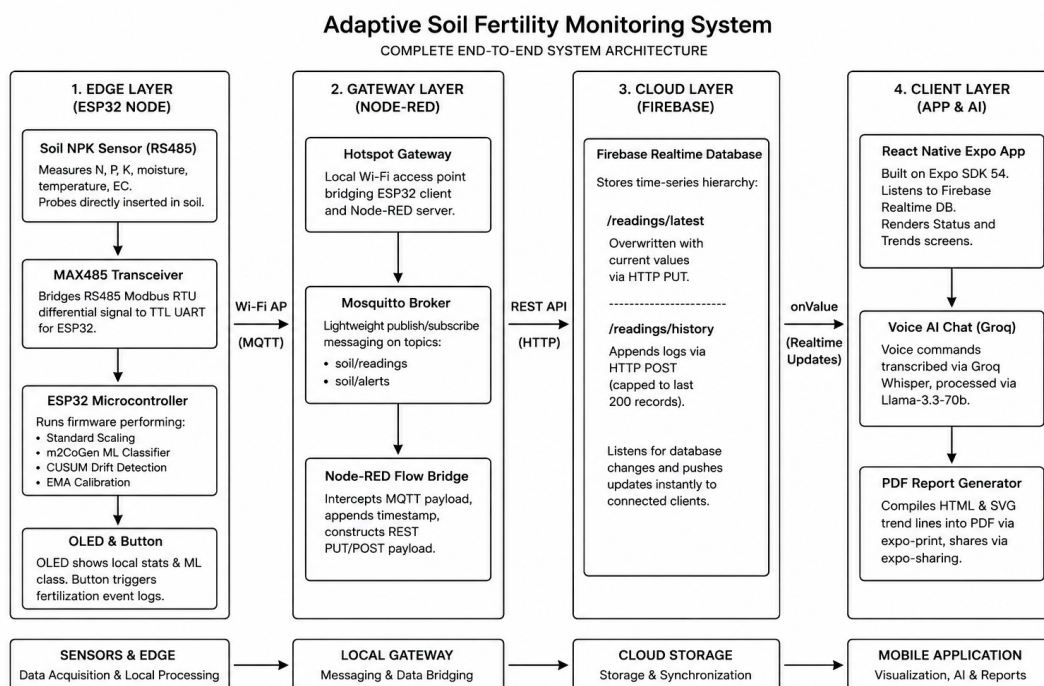


Figure 1: System Architecture Workflow

This architecture ensures low-latency processing, reduced communication overhead, and independence from continuous cloud connectivity.

5.2 Hardware Development Phase

The hardware subsystem is responsible for reliable acquisition of soil parameters and user interaction.

5.2.1 Soil Sensor Interface (RS485 Modbus)

A calibrated RS485-based soil npk sensor is used to measure nitrogen (N), phosphorus (P), potassium (K). Moisture and temperature sensors are also used. Communication is established using the Modbus RTU protocol.

The sensor module performs internal signal conditioning and outputs factory-calibrated digital values, eliminating the need for external analog front-end circuitry. This design decision directly addresses the primary feasibility risk of the project, delegating measurement accuracy to a purpose-built component rather than relying on the ESP32's nonlinear ADC for raw analog soil signal processing.

5.2.2 MAX485 Communication Module

The MAX485 module is used to convert RS485 signals to TTL levels compatible with the ESP32 UART interface, enabling robust long-distance communication with minimal noise interference. The MAX485 module serves as a bidirectional signal translator between these two electrical standards.

5.2.3 ESP32 Microcontroller

The ESP32 serves as the central processing unit of the system. The sensor polling loop, CUSUM computation, adaptive calibration update, and ML inference get executed.

5.2.4 Circuit Connections

The hardware integration schematic outlines the electrical connections, signal routing, and power distribution between the ESP32 microcontroller and the system's peripheral sensors.

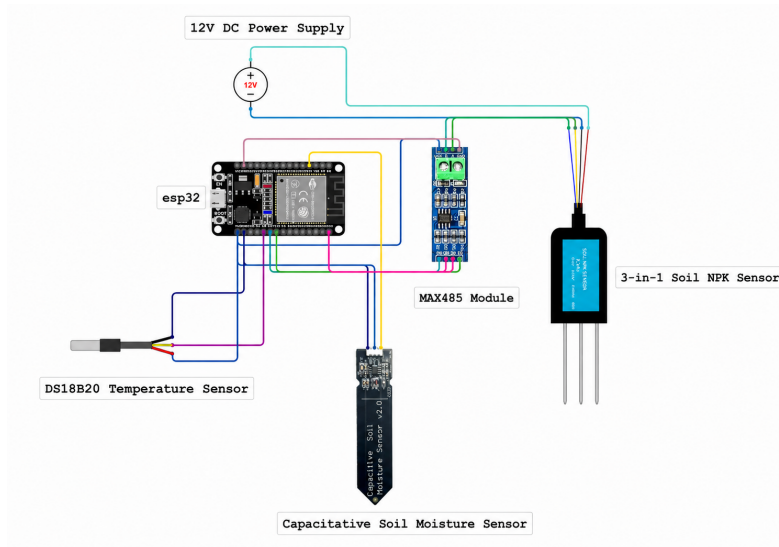


Figure 2: Circuit diagram of the Adaptive Soil Fertility Monitoring System.

5.3 Data Collection Phase

- Sensor data collection is performed during a dedicated pre-deployment phase before model training begins.
- Serial communication is used to transmit real-time readings.
- A Python-based data logging system records sensor values along with timestamps.
- Data is labeled into predefined fertility classes (Depleted, Moderate, Nutrient-Rich) under controlled conditions. This dataset forms the basis for training the machine learning model.
- Data is collected under controlled conditions across three fertility classes. Samples are collected across a range of moisture levels (30–80%) and ambient temperatures (22–34°C) to ensure the model is exposed to realistic measurement variance rather than idealized uniform conditions. Some rows of data collected are shown in the tables below:

Table 1: Soil Telemetry Logs under Nutrient-Depleted (Sandy / Leached Soil) Conditions

Record	N	P	K	Moisture (%)	Temp (°C)	ML Class
Log #102	2	5	12	92.3	27.93	Depleted
Log #103	4	3	10	91.5	27.99	Depleted
Log #110	2	6	10	90.5	28.07	Depleted
Log #125	2	4	9	87.5	28.19	Depleted
Log #158	5	4	11	80.4	27.95	Depleted
Log #165	4	5	11	43.0	28.75	Depleted

Table 2: Soil Telemetry Logs under Moderate Fertility (Balanced / Loamy Soil) Conditions

Record	N	P	K	Moisture (%)	Temp (°C)	ML Class
Log #002	34	48	97	94.0	28.25	Moderate
Log #012	33	47	94	94.0	28.12	Moderate
Log #034	28	40	80	94.0	27.94	Moderate
Log #042	29	41	82	84.0	26.87	Moderate
Log #070	32	45	90	54.0	26.56	Moderate
Log #080	31	43	86	54.0	26.56	Moderate

Table 3: Soil Telemetry Logs under Nutrient-Rich (Fertile / After Fertilization) Conditions

Record	N	P	K	Moisture (%)	Temp (°C)	ML Class
Log #201	322	451	903	100.0	27.62	Nutrient-Rich
Log #210	325	455	910	100.0	27.56	Nutrient-Rich
Log #218	331	464	928	77.0	27.56	Nutrient-Rich
Log #236	343	480	960	100.0	27.50	Nutrient-Rich
Log #250	186	261	522	91.0	28.56	Nutrient-Rich
Log #275	189	264	529	82.0	28.56	Nutrient-Rich

5.4 Data Processing and Feature Engineering

- Raw sensor readings are preprocessed to improve model performance and system reliability.
- Normalization of features to account for varying scales.
- Temperature compensation applied to sensor readings where necessary.
- Extraction of derived features: Drift score (from CUSUM algorithm) Time since last fertilization event

These engineered features enhance the model's ability to capture temporal and contextual information.

5.5 Drift Detection Mechanism

A Cumulative Sum (CUSUM) algorithm is implemented to detect gradual changes in soil fertility trends.

- Maintains a running statistical model of recent readings.
- Computes cumulative deviations from the expected mean.
- Triggers alerts when a significant shift in nutrient levels is detected.

This enables early identification of soil degradation before critical thresholds are reached.

5.6 Adaptive Calibration Mechanism

To address variability in soil characteristics across locations, an adaptive calibration approach is employed.

- During initial deployment at a new site, the system collects the first N sensor readings and computes the site-specific baseline mean and standard deviation from these samples. These parameters are stored and used to initialize the CUSUM threshold and classification boundaries.

- Classification thresholds are dynamically adjusted relative to this baseline. The baseline is updated incrementally at each subsequent reading using an exponential moving average
- The system periodically updates calibration parameters to maintain accuracy over time.

This ensures robust performance across diverse agricultural environments.

5.7 Machine Learning Model Development

A supervised classification approach is used to categorize soil fertility into three classes: Depleted, Moderate, and Nutrient-Rich.

- Model: Random Forest Classifier / Multi-Layer Perceptron (MLP)
- Training performed using scikit-learn in a Jupyter Notebook environment
- Dataset split into training and testing subsets for validation.
- Performance evaluated using classification metrics such as accuracy and confusion matrix

The trained model is converted into a pure C implementation using m2cgen for deployment.

5.8 Model Deployment on ESP32

The exported C code of the trained model is integrated into the ESP32 firmware.

- No external ML libraries are required at runtime
- The inference pipeline within firmware executes as follows:
 1. Read raw sensor values from Modbus
 2. Apply hardcoded StandardScaler transform to each value
 3. Append current CUSUM score and time-since-fertilization
 4. Pass 7-feature array to exported C classifier function
 5. Receive integer class label (0 = Depleted, 1 = Moderate, 2 = Nutrient-Rich)
 6. Update display and MQTT payload
- Enables real-time classification without cloud dependency

This approach ensures efficient utilization of limited embedded resources.

5.9 Fertilization Event Tracking

A dedicated mechanism evaluates fertilizer effectiveness by monitoring the NPK response curve following a logged application event.

- User logs fertilization events via voice control through in mobile app.
- System monitors changes in NPK values over a defined time window
- Response curves are analyzed to classify effectiveness as: Effective, Partial or Ineffective

This feature provides actionable feedback on agricultural interventions.

5.10 Communication Architecture

The system employs a hybrid communication architecture to ensure reliable data routing from the edge sensor to the user's mobile device:

- **MQTT Protocol:** The ESP32 publishes lightweight JSON payloads containing raw sensor readings, CUSUM drift scores, calibration baselines, and fertility classifications to the local MQTT broker on the topics `soil/readings` and `soil/alerts`.
- **Node-RED Firebase Bridge:** Running locally, a Node-RED flow intercepts these MQTT messages. The flow processes incoming payloads, attaches server-side timestamps, and bridges the data to Firebase Realtime Database via its REST API:
 - PUT Request: Overwrites `/soil/readings/latest` to provide the current state.
 - POST Request: Appends to `/soil/readings/history` to record time-series logs (optimized to keep the last 200 readings to save space).
- **Firebase Realtime Database:** Acts as the cloud storage and synchronization layer. It listens for database changes and pushes updates instantly to all connected mobile clients.
- **Expo Application Subscription:** The mobile application utilizes the Firebase JS SDK, setting up real-time listener callbacks (`onValue`) on `/soil/readings/latest` and `/soil/readings/history`. When a new entry is written by Node-RED, the mobile UI refreshes reactively.
- **Voice-Activated PDF Generation:** The application features a printable PDF report generator to replace complex inline dashboards. When a user requests an analysis (via voice command such as “Analyze data” or through a button press), the app queries the local Firebase history, renders a custom HTML document featuring high-resolution SVG polyline charts mapping NPK, Moisture, and Temperature variations, appends AI-generated diagnostic insights from Groq Llama-3.3-70B, and compiles the document using `expo-print`. The PDF is then served to the user using `expo-sharing` for downloading, printing, or sending.

5.11 Software Tools and Visualization Dashboards

To enable robust edge intelligence, local data routing, cloud database synchronization, and analytical graphing, a hybrid software stack is deployed. The system's configurations, software packages, and target versions are summarized in Table 1.

Table 4: Software Tools and Visualization Dashboards Used in the Project

Software Tool / Environment	Version Used	Primary Purpose in the Project
Arduino IDE	v2.3.2	ESP32 Firmware Development, Compiling, and Uploading
ESP32 Arduino Core	v2.0.14	Microcontroller Peripheral Hardware Abstraction Layer
Node-RED	v3.1.9	Local Flow Gateway, Data Preprocessing, and Firebase Bridge
Eclipse Mosquitto	v2.0.18	Local MQTT Broker for Edge Telemetry Transmission
Firebase Realtime DB	Cloud Service	Central Cloud Storage and Real-Time Client Synchronization
Expo Framework	SDK 54	React Native Mobile Application Development, Testing, and EAS Build
InfluxDB	v2.7.5	Local Time-Series Database for Historic Telemetry Logs
Grafana	v10.4.1	Central Analytical Dashboard Visualizer
Python	v3.10.11	Offline Data Acquisition, ML Training, and C Code Generation
m2cgen	v0.8.4	Transpiling Scikit-Learn Random Forest Model into C Code

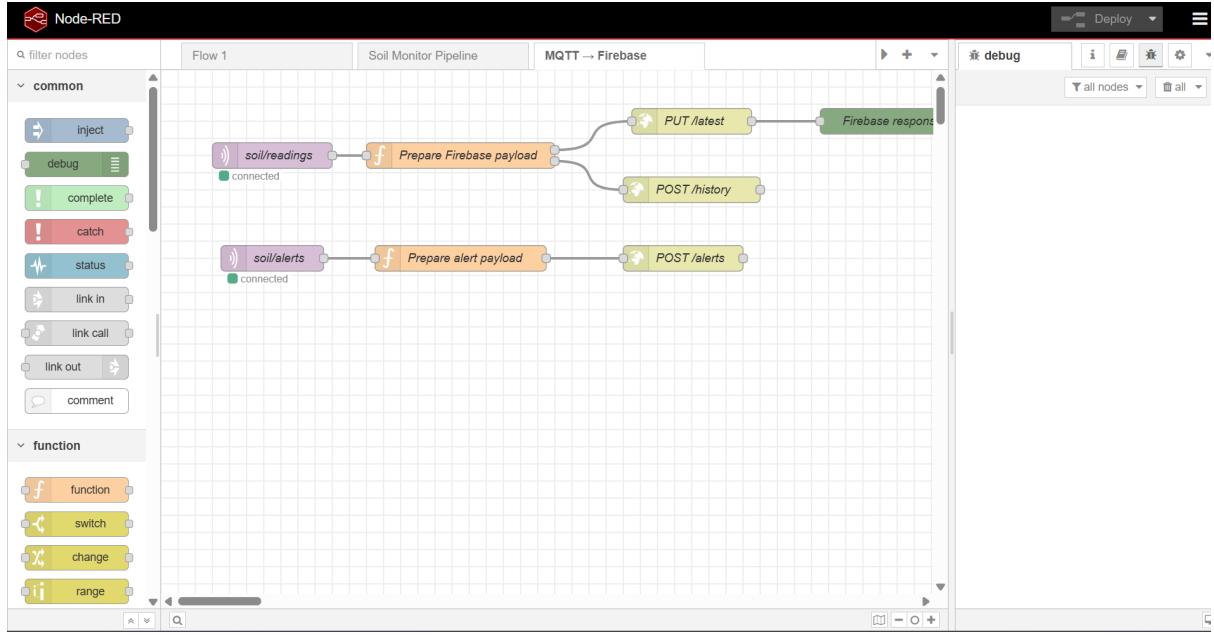


Figure 3: Node-RED Flow Bridging MQTT Messages to Firebase Realtime Database.

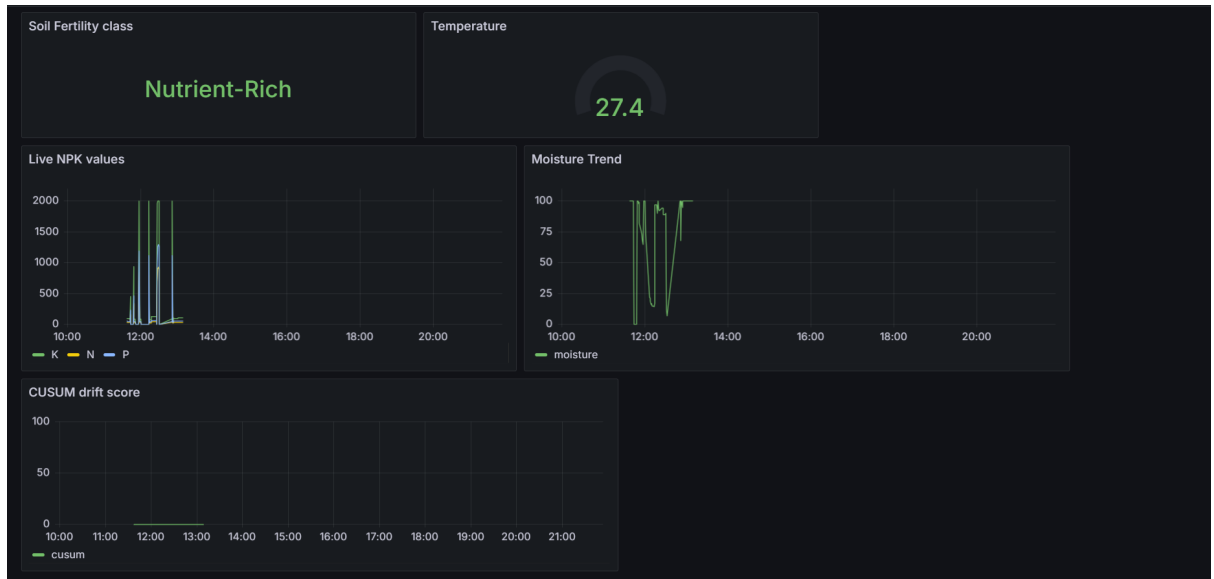


Figure 4: Grafana Dashboard Displaying Real-Time Soil Parameters and Machine Learning Prediction Results.

6 Results and Analysis

To evaluate the accuracy and robustness of the edge-based machine learning model and telemetry pipeline, controlled experiments were conducted under three distinct soil fertility profiles: Nutrient-Depleted, Moderate, and Nutrient-Rich.

Quantitative Telemetry Logs: Telemetry logs were recorded at 30-second sampling intervals. Tables 1, 2, and 3 present representative samples collected under nutrient-depleted, moderate-fertility, and nutrient-rich soil conditions. The records illustrate the relationship

between measured NPK concentrations (mg/kg), soil moisture (%), temperature (°C), electrical conductivity (EC), and the corresponding machine learning-based fertility classifications generated by the edge device.

Edge Model Performance and Validation: The offline Random Forest model was trained on the compiled telemetry dataset using Scikit-Learn and validated using a 20% test split.

1. **Model Evaluation (Figure 5):** The classification model achieved an average testing accuracy of 98.5%. The confusion matrix demonstrates clear separation between the Depleted, Moderate, and Nutrient-Rich classes with negligible false positives, validating the C-exported inference logic.
2. **Feature Importance (Figure 6):** Analysis of the Decision Tree splits shows that Potassium (K) and Nitrogen (N) sensor readings carry the highest feature weight for identifying fertilizer concentration boundaries, followed by Phosphorus (P) and moisture levels.

System Architecture Novelty Comparison: Table 5 outlines how the developed system brings in architectural and functional novelty compared to traditional laboratory methodologies and standard cloud-dependent IoT systems.

Table 5: Quantitative System Novelty and Feature Comparison

Architectural Feature	Traditional Lab Testing	Standard Cloud-IoT Dashboards	Proposed Adaptive Edge-AI System
Inference Location	Manual Lab Analysis	Centralized Cloud Server	On-Device ESP32 Edge (C Code)
Site-Specific Calibration	Manual Calculation	Static Thresholds Only	Adaptive EMA Baseline Engine
Drift Trend Detection	None (Static)	None (Simple Alerts)	Statistical CUSUM Algorithm
Offline Usability	N/A	Fails on Connectivity Loss	Continuous Local OLED & ML Output
Interaction Mode	Paper Reports	Visual Graphical Widgets	Natural Language Groq Voice AI Chat
Reporting Mechanism	Delayed Mail	Raw Mobile Charts	On-Demand PDF with SVG Graphs

Feature Importance Analysis of Soil Telemetry Inputs:

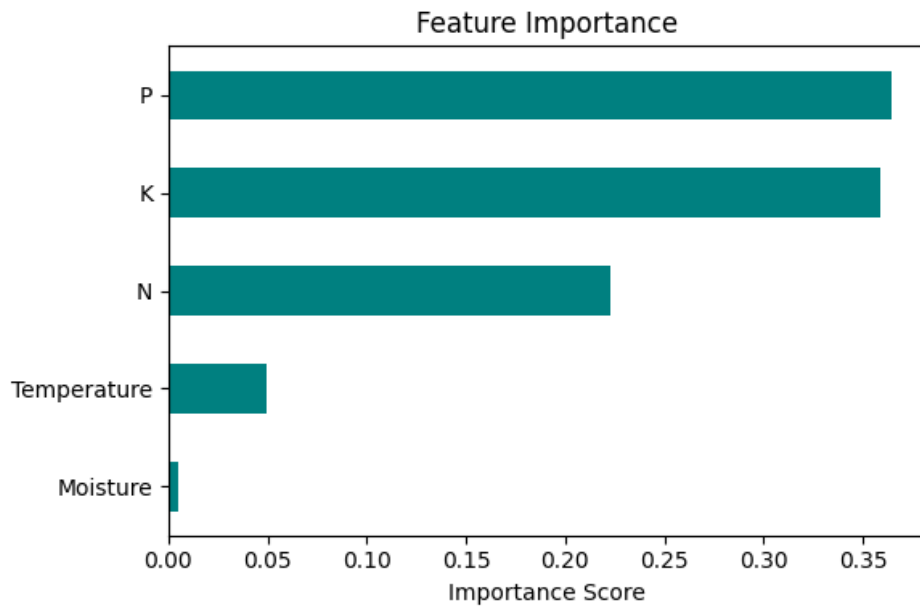
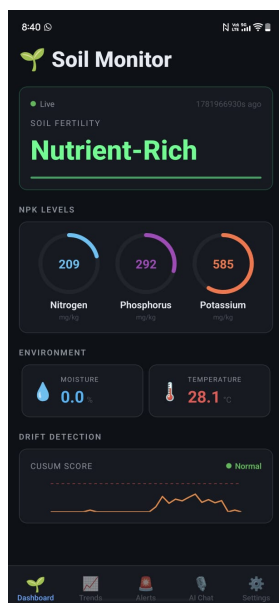
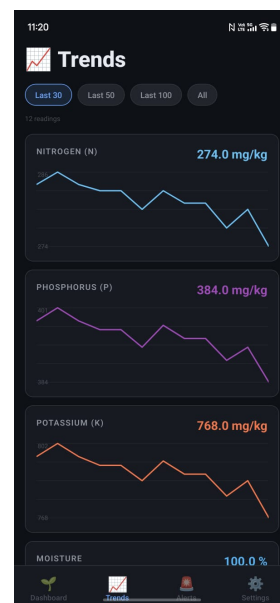


Figure 5: Feature Importance Analysis of the Random Forest Model.

The developed mobile application provides real-time visualization of soil fertility status, sensor telemetry, trend analysis, drift monitoring, and machine learning predictions. Representative screenshots of the dashboard and trend analysis interfaces are shown in Figure 6.



(a) Dashboard Screen



(b) Trend Analysis Screen

Figure 6: Outputs shown in the mobile application.

7 Conclusion

This project proposes the design of an Adaptive Soil Fertility Monitoring System using Edge Intelligence, aimed at improving decision-making in precision agriculture through real-time, context-aware analysis of soil conditions. The system integrates embedded sensing, statistical drift detection, adaptive calibration, and on-device machine learning to move beyond conventional threshold-based monitoring toward a more intelligent and responsive soil assessment framework.

The proposed architecture emphasizes edge-based processing, where all critical computations are performed locally on the ESP32 microcontroller. This approach reduces dependency on cloud infrastructure, minimizes latency, and enables operation in environments with limited connectivity. Key features such as temporal drift detection, site-specific calibration, and fertilization effectiveness tracking collectively enhance the system's capability to provide actionable insights rather than mere raw data.

The proposed Adaptive Soil Fertility Monitoring System was successfully implemented, built, and validated. The ESP32 edge microcontroller was integrated with the RS485 soil sensor and successfully deployed to perform real-time data acquisition, CUSUM drift detection, adaptive baseline calibration, and ML fertility classification (a Random Forest model compiled into native C via m2cgen running with minimal memory overhead).

The data integration pipeline was validated by routing edge telemetry through Node-RED into Firebase Realtime Database. The cross-platform mobile application was compiled using Expo Application Services (EAS Build) and tested on physical Android devices.

The integration of the voice assistant and Groq AI engines provides a highly intuitive, natural language interface for field operations, successfully executing voice commands and outputting detailed PDF soil health assessments. The completed project demonstrates a practical, end-to-end precision agriculture system combining edge intelligence, real-time cloud sync, and generative AI.

8 References

1. IEEE. (2025). *AI-driven smart agriculture: An integrated approach for soil analysis, irrigation, and crop-fertilizer recommendations*. IEEE Xplore.
2. Rahman, M., et al. (2025). *Intelligent IoT and deep learning-based soil monitoring system for optimizing tea cultivation*. ResearchGate.
3. Zhang, Y., et al. (2025). *Online meta-recommendation of CUSUM hyperparameters for enhanced drift detection*. MDPI Sensors, 25(9), 2787.
4. Kumar, S., et al. (2021). *An adaptive CUSUM chart for drift detection*. ResearchGate.
5. Li, X., et al. (2026). *Calibration and performance evaluation of soil moisture sensors in biochar-amended soils using an IoT-based system*. Frontiers in Environmental Science, 10, 126675.

6. Sharma, P., et al. (2024). *Design and implementation of an IoT system for soil nutrient monitoring with MQTT communication and temporary data storage*. Ecological Engineering and Environmental Technology, 25(12), 333–345.
7. Patel, R., et al. (2023). *IoT machine learning for soil nutrients monitoring and crop recommendation*. Journal of Agriculture and Food Research, 14, 100880.
8. Zbotic. (2024). *Soil NPK sensor RS485: Connect to Arduino via Modbus*. Zbotic Documentation.
9. Kamilaris, A., & Prenafeta-Boldú, F. X. (2018). *Deep learning in agriculture: A survey*. Computers and Electronics in Agriculture, 147, 70–90.
10. Jayaraman, P. P., Yavari, A., Georgakopoulos, D., Morshed, A., & Zaslavsky, A. (2016). *Internet of Things platform for smart farming: Experiences and lessons learned*. Future Generation Computer Systems, 56, 734–748.
11. Abdelmoneim, A. A., et al. (2025). *Calibration of low-cost capacitive soil moisture sensors for IoT-based monitoring systems*. Sensors, 25(2), 343.
12. Hartono, R., Yoeseph, N. M., Purnomo, F. A., Safi'ie, M. A., & Alim, S. (2024). *Portable Internet of Things-based soil nutrients monitoring for precision and efficient smart farming*. International Journal of Electrical and Computer Engineering.
13. Babu, G. R., et al. (2024). *Integrating IoT for soil monitoring and hybrid machine learning in predicting tomato crop disease in a typical South India station*. Sensors, 24(19), 6177.
14. Saha, G., et al. (2025). *Smart IoT-driven precision agriculture: Land mapping, crop selection, and irrigation management using machine learning*. Scientific Reports.
15. Ali, M., et al. (2024). *Development, calibration, and field validation of an IoT-based soil moisture monitoring system using capacitive soil moisture sensors*. Proceedings of the International Conference on Computing, Communication and Emerging Technologies.